

Tema 3.4: Captura Centralizada de Excepciones y Middleware de Errores HTTP

Resumen

Este tema aborda el diseño e implementación de un mecanismo centralizado para la captura, procesamiento y formateo de excepciones en aplicaciones web empresariales construidas con Spring Boot. Se analiza cómo desacoplar la lógica de manejo de errores de la capa de controladores utilizando la anotación `@RestControllerAdvice` y la especificación estándar RFC 7807 (Problem Details), garantizando respuestas HTTP homogéneas, seguras y libres de fugas de información interna del servidor.

Concepto Principal (El 'Qué')

En arquitecturas distribuidas y APIs RESTful, el **Manejo Centralizado de Excepciones** (Centralized Exception Handling) es un patrón de diseño arquitectónico que intercepta cualquier anomalía o condición excepcional producida durante la ejecución de una solicitud HTTP, transformándola en una respuesta estructurada con un código de estado HTTP adecuado (2xx, 4xx, 5xx).

Sin una estrategia centralizada, el código fuente se contamina con bloques `try-catch` repetitivos en los controladores REST y servicios de negocio, incumpliendo el *Principio de Responsabilidad Única (SRP)*. Además, delegar el manejo de errores por defecto a los motores de servlet (como Apache Tomcat) expone la traza de pila interna (*stack trace*) al cliente, lo cual constituye una vulnerabilidad crítica de divulgación de información según el estándar **OWASP Top 10**.

Mediante **Spring Boot**, el framework proporciona el patrón Aspect-Oriented Programming (AOP) a través de la anotación estereotipo `@RestControllerAdvice` (o `@ControllerAdvice`), actuando como un *interceptor de excepciones global* que captura excepciones lanzadas por cualquier controlador en el contexto de la aplicación.

Analogía (La Intuición)

Considere el departamento de atención al cliente en un hospital de alta complejidad. Cuando un médico, un especialista o un laboratorista detecta una anomalía en un procedimiento interno (una excepción en la capa de negocio o datos), ellos no abandonan sus funciones para redactar una carta formal de notificación y entregarla personalmente al paciente en la sala de espera.

En su lugar, el personal médico eleva un reporte de alerta estandarizado a la **Central de Comunicación Médica** (el `@RestControllerAdvice`). Esta central procesa el reporte interno, elimina la jerga técnica confusa o alarmante, y emite un comunicado oficial unificado y respetuoso hacia el paciente con el código de clasificación apropiado (por ejemplo, *"Examen pendiente por muestra insuficiente"*), protegiendo la confidencialidad de los protocolos internos de la institución.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación se presenta la implementación completa de un sistema de captura centralizada de errores en Java 21 con Spring Boot 3.x, haciendo uso del estándar **RFC 7807 (Problem Details)**:

1. Definición del Objeto DTO para Respuestas de Error

```
package cr.ac.ucr.paraíso.ie.expresofast.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import java.time.LocalDateTime;
import java.util.Map;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponseDTO {
    private String title;
    private int status;
    private String detail;
    private String instance;
    private LocalDateTime timestamp;
    private Map<String, String> invalidFields;

    public ErrorResponseDTO(String title, int status, String detail,
```

```

        String instance) {
    this.title = title;
    this.status = status;
    this.detail = detail;
    this.instance = instance;
    this.timestamp = LocalDateTime.now();
}

public ErrorResponseDTO(String title, int status, String detail,
        String instance,
        Map<String, String> invalidFields) {
    this(title, status, detail, instance);
    this.invalidFields = invalidFields;
}

public String getTitle() { return title; }
public int getStatus() { return status; }
public String getDetail() { return detail; }
public String getInstance() { return instance; }
public LocalDateTime getTimestamp() { return timestamp; }
public Map<String, String> getInvalidFields() {
    return invalidFields;
}
}

```

2. Excepción Personalizada de Dominio

```

package cr.ac.ucr.paraíso.ie.expresofast.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}

```

3. Middleware Centralizado de Excepciones (GlobalExceptionHandler)

```

package cr.ac.ucr.paraíso.ie.expresofast.exception;

import cr.ac.ucr.paraíso.ie.expresofast.dto.ErrorResponseDTO;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;

```

```

import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponseDTO> handleResourceNotFound(
        ResourceNotFoundException ex,
        HttpServletRequest request) {

        ErrorResponseDTO error = new ErrorResponseDTO(
            "Recurso No Encontrado",
            HttpStatus.NOT_FOUND.value(),
            ex.getMessage(),
            request.getRequestURI()
        );
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponseDTO> handleValidationErrors(
        MethodArgumentNotValidException ex,
        HttpServletRequest request) {

        Map<String, String> errors = new HashMap<>();
        for (FieldError fieldError : ex.getBindingResult().getFieldErrors()) {
            errors.put(fieldError.getField(), fieldError.getDefaultMessage());
        }

        ErrorResponseDTO error = new ErrorResponseDTO(
            "Error de Validación de Datos",
            HttpStatus.BAD_REQUEST.value(),
            "Uno o más campos de la solicitud no cumplen las restricciones.",
            request.getRequestURI(),
            errors
        );
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }
}

```

```

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponseDTO> handleGenericException(
        Exception ex,
        HttpServletRequest request) {

        ErrorResponseDTO error = new ErrorResponseDTO(
            "Error Interno del Servidor",
            HttpStatus.INTERNAL_SERVER_ERROR.value(),
            "Ocurrió un error inesperado. Por favor contacte al soporte.",
            request.getRequestURI()
        );
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

Explicación Paso a Paso del Código:

1. **@RestControllerAdvice**: Anota la clase `GlobalExceptionHandler` indicándole a Spring que intercepte de forma global las excepciones generadas en cualquier controlador anotado con `@RestController`.
2. **@ExceptionHandler(ResourceNotFoundException.class)**: Captura específicamente las excepciones de negocio lanzadas cuando no se encuentra una entidad. Convierte la excepción en una respuesta `404 Not Found` amigable.
3. **@ExceptionHandler(MethodArgumentNotValidException.class)**: Intercepta los errores de validación desencadenados por `@Valid` en los DTOs de entrada. Extrae cada `FieldError` del `BindingResult` y construye un mapa campo-mensaje de error.
4. **@ExceptionHandler(Exception.class)**: Actúa como un *filtro de captura general* (catch-all) para excepciones no controladas (como fallos de base de datos o nulos). Evita exponer el `stacktrace` interno devolviendo un código `500 Internal Server Error`.

Trampas Comunes

Fuga de Stack Traces en Entornos Productivos:

- *Error*: No capturar la excepción raíz `Exception.class` ni configurar `server.error.include-stacktrace=never` en `application.properties`, permitiendo

que el cliente visualice clases internas, nombres de tablas y consultas SQL.

- *Solución:* Retornar únicamente mensajes sanitizados mediante DTOs y registrar la traza técnica detallada exclusivamente en los logs del servidor mediante un logger (`LoggerFactory.getLogger()`).

Códigos de Estado HTTP Incorrectos:

- *Error:* Retornar un código HTTP 200 OK en el encabezado pero incluir en el cuerpo de la respuesta un objeto JSON con `{"status": 500, "error": "Internal Error"}`.
- *Solución:* Asegurar que el `ResponseEntity` devuelva el código de estado HTTP real correspondiente tanto en los encabezados del protocolo HTTP como en el cuerpo del JSON.

Captura Excesivamente Broad en Capas Internas:

- *Error:* Envolver la lógica de negocio dentro de los `@Service` en bloques try-catch (`Exception e`) que tragan la excepción y retornan `null` o cadenas vacías.
- *Solución:* Permitir que las excepciones fluyan hacia la capa superior o relanzar excepciones de dominio personalizadas para que sean interceptadas centralizadamente por el `@RestControllerAdvice`.

Pregunta de Reflexión

En una arquitectura de microservicios distribuida donde un servicio invoca a otros tres mediante llamadas HTTP REST: ¿De qué manera la captura centralizada de errores debe manejar las excepciones de tiempo de espera (*Timeouts*) y de red para evitar fallos en cascada y mantener la resiliencia del sistema?